

Race Conditions

Definition

A **race condition** occurs when two or more processes or threads access shared data at the same time, and the final result depends on the exact timing or order of execution.

1. Why Race Conditions Happen

Main Idea

Processes often cooperate by sharing storage. The shared storage may be in main memory, in a kernel data structure, or in a shared file. The location does not change the core problem: multiple processes can read and write the same data.

Key Message

Sharing data is not automatically wrong. The danger appears when several processes access the same shared value without coordination.

When the operating system switches the CPU from one process to another, the first process may stop in the middle of a multi-step operation. If another process changes the same shared data before the first process resumes, the first process may continue using stale information.

Practical Mental Model

The sequence “read a shared value, use it, then write it back” is not automatically one indivisible action. Another process may run between those steps.

2. Print Spooler Example

Scenario

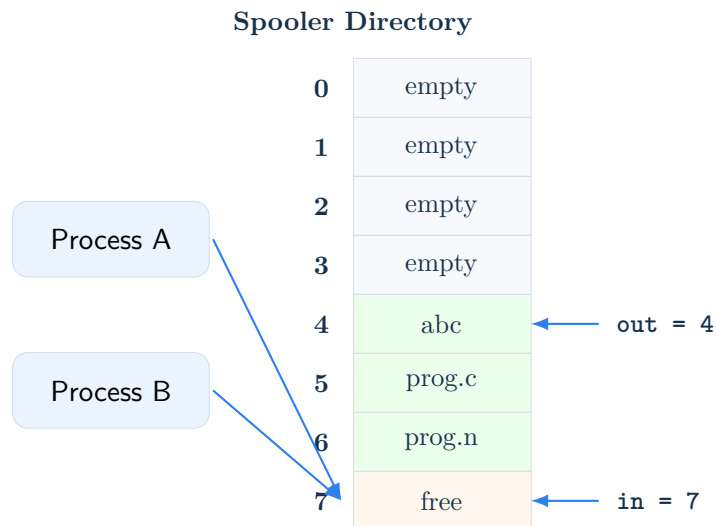
When a process wants to print a file, it places the file name in a shared spooler directory. A printer daemon later checks that directory, prints queued files, and removes their names.

The spooler directory contains numbered slots. Two shared variables control the queue:

- **out**: points to the next file that should be printed.
- **in**: points to the next free slot where a new file name can be placed.

Initial State

Slots 0–3 are empty because their files have already been printed. Slots 4–6 are full. Slot 7 is the next free slot, so **out** = 4 and **in** = 7.



3. How the Race Occurs

The Timing Problem

Processes A and B both decide to print a file at nearly the same time. Both need to read **in**, use the slot number, write a file name, and then update **in**.

1. Process A reads **in** and stores 7 in its local variable **next_free_slot**.
2. A clock interrupt occurs before A writes its file name.
3. The CPU switches to process B.
4. Process B reads **in**. It also sees 7.
5. Process B writes its file name into slot 7 and updates **in** to 8.
6. Process A resumes and still believes slot 7 is free.
7. Process A writes its file name into slot 7, overwriting B's file name.
8. Process A updates **in** to 8.

Result

The spooler directory still looks internally consistent because **in** = 8. However, process B's print request has silently disappeared.

4. Timeline View

Step	Process A	Process B
1	Reads <code>in</code> = 7.	–
2	Gets interrupted before writing.	–
3	–	Reads <code>in</code> = 7.
4	–	Writes file name into slot 7.
5	–	Sets <code>in</code> = 8.
6	Resumes with old local value 7.	–
7	Writes file name into slot 7, replacing B's entry.	–
8	Sets <code>in</code> = 8.	B's request is lost.

5. Critical Section

Critical Section

A **critical section** is a part of a program that accesses shared data and must not be executed by more than one process or thread at the same time.

For the print spooler, the dangerous sequence is:

```
next_free_slot = in
spooler_directory[next_free_slot] = file_name
in = next_free_slot + 1
```

Core Lesson

The problem is not any single line by itself. The problem is that the whole read-modify-write sequence must behave like one protected operation.

6. Why Race Conditions Are Hard to Debug

Timing Dependence

Race conditions are difficult because the bug depends on scheduling. The same program may work correctly in many test runs, then fail when the CPU switches processes at an unlucky moment.

- The failure may be rare.
- Re-running the same program may make the bug disappear.
- Adding print statements or logs may change timing and hide the bug.
- More CPU cores increase the chance of true parallel overlap.

Why the Spooler Bug Is Subtle

The printer daemon may not detect anything wrong. The queue still has a valid-looking slot 7 and a valid-looking `in` value. The damage is logical: one user's request was overwritten.

7. How Operating Systems Prevent Race Conditions

Synchronization

Synchronization means using rules or mechanisms that control how processes or threads enter critical sections and access shared data.

Mechanism	Purpose
Locks / mutexes	Allow only one process or thread to enter a critical section at a time.
Semaphores	Control access to shared resources using counters.
Atomic operations	Perform small read-modify-write operations indivisibly.
Monitors	Combine shared data with safe procedures for accessing it.
Message passing	Avoid shared writable data by communicating through messages.

8. Quick Review

Key Points

- A race condition occurs when the result depends on the order and timing of concurrent access.
- Shared storage may be memory, a kernel data structure, or a file.
- In the spooler example, processes A and B both believe slot 7 is free.
- Process A overwrites process B's file name, so B's print request is lost.
- The queue can still look internally consistent, making the bug hard to notice.
- The dangerous code region is called a critical section.
- Race conditions are prevented using synchronization mechanisms such as locks, semaphores, atomic operations, monitors, or message passing.